

ASSIGNMENT-19.2

Task 1 – Python to Java Conversion

Task:

Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Instructions:

- Prompt AI to translate a Python script that reads from and writes to a file into Java.
- Ensure proper handling of file streams, exceptions, and input/output formatting.
- Verify that the Java program produces the same file content and console output as the Python version.
- Test both implementations with sample input files.

Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.



The screenshot displays a code editor with a Java program named `FileHandler.java`. The program imports `java.io.BufferedReader`, `java.io.FileReader`, `java.io.FileWriter`, and `java.io.IOException`. It defines a `FileHandler` class with a `main` method that reads from `input.txt` using a `BufferedReader` and `FileReader`, appends each line to a `StringBuilder`, and then prints the concatenated content to the console. The code includes a `try-catch` block for `IOException` and a `return` statement.

On the right side of the screenshot, there is a chat interface titled "PYTHON TO JAVA FILE HANDLING CON...". It contains the following text:

Convert the following Python file handling program into Java.

Requirements:

- Use appropriate Java file handling classes (`FileReader`, `BufferedReader`, `FileWriter`, etc)
- Handle exceptions using `try-catch`
- Maintain the same functionality and output formatting

Also provide:

1. Explanation of the Java code
2. Differences between Python and Java file handling
3. Sample input and output for verification

Python Code:

OUTPUT:

```
HELLO WORLD
JAVA FILE HANDLING
PROGRAMMING CONCEPTS
PROCESSING TEXT FILES
```

Task 2 – C++ to Python Function Conversion

Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

- Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).
- Ask AI to generate an equivalent Python implementation.
- Ensure logical correctness and validate results using multiple test cases.
- Confirm that the Python version maintains the same sorting behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.

```
task2.py > bubble_sort
1 # C++ to Python Sorting Algorithm Conversion
2 # Original: Bubble Sort in C++
3 # Converted to: Pythonic Bubble Sort Implementation
4
5 def bubble_sort(arr):
6     """
7     Sorts an array using the Bubble Sort algorithm.
8
9     Bubble Sort repeatedly steps through the list, compares adjacent elements,
10    and swaps them if they are in the wrong order.
11
12    Args:
13        arr (list): The list to be sorted
14
15    Time Complexity:
16        - Best Case: O(n) - when array is already sorted (with optimization)
17        - Average Case: O(n²)
18        - Worst Case: O(n²) - when array is reverse sorted
19
20    Space Complexity: O(1) - sorts in-place, no extra space needed
21
22    Returns:
23        list: The sorted array (modifies in-place)
24    """
25    n = len(arr)
```

```
← PYTHON TO JAVA FILE HANDLING CON...
Convert the following C++ sorting algorithm into Python.

Requirements:
• Maintain the same logic and sorting behavior
• Use clean and Pythonic syntax

Also provide:
1. Step-by-step explanation
2. Time complexity analysis
3. At least 3 test cases with outputs

C++ Code:
#include <iostream>
using namespace std;

int main() {
    int arr[] = {5, 2, 9, 1, 5, 6};
    int n = 6;

    for(int i = 0; i < n-1; i++)
```

OUTPUT:

```

Test Case 6 - Single Element:
Original array: [42]
Sorted array: [42]

Test Case 7 - Optimized Bubble Sort (Already Sorted):
Original array: [1, 2, 3, 4, 5]
Sorted array: [1, 2, 3, 4, 5]
(Optimized version exits early since no swaps needed)

```

Task 3 – SQL to Pandas Query

Task:

Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

- Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).
- Prompt AI to generate equivalent Pandas code using `groupby()` and aggregation functions.
- Test the generated code using a sample dataset.
- Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.

```

task3.py > ...
1 import pandas as pd
2 import io
3
4 # SQL Equivalent: SELECT department, SUM(sales) AS total_sales FROM sales_data GROUP BY department;
5
6 # =====
7 # METHOD 1: Using groupby() with sum() - Most Pythonic and Commonly Used
8 # =====
9
10 def sql_groupby_groupby_agg(data):
11     """
12     Convert SQL GROUP BY using pandas groupby() method.
13
14     SQL: SELECT department, SUM(sales) AS total_sales
15         FROM sales_data
16         GROUP BY department;
17
18     This is the most common and recommended approach.
19     """
20     result = data.groupby('department')['sales'].sum().reset_index()
21     result.columns = ['department', 'total_sales']
22     return result
23
24 # =====
25 # METHOD 2: Using groupby() with agg() - More Explicit
26 # =====
27
28

```

PYTHON TO JAVA FILE HANDLING CON...

Convert the following SQL query into equivalent Pandas code.

Requirements:

- Use pandas DataFrame
- Apply groupby() and aggregation functions correctly
- Ensure the output matches the SQL query result

Also provide:

1. Explanation of the Pandas code
2. Sample dataset
3. Output comparison

SQL Query:

```

SELECT department, SUM(sales) AS
total_sales
FROM sales_data
GROUP BY department;

```

Sample Data:

department	sales
A	100
B	200

OUTPUT:

```
Method 1 == Method 2: True
Method 1 == Method 3: True
Method 1 == Method 4: True

✓ All methods produce identical output!
```

Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Scenario:

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

- Converting a REST API implementation from Python (Flask) to Java (Spring Boot).
- Translating a data processing script from Python to C++ for performance optimization.

Instructions:

- Provide AI with source code in one programming language.
- Prompt AI to convert the code into another chosen language.
- Execute and test both versions to ensure identical functionality.
- Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

```
J DemoApplication.java > DemoApplication
1 package com.example.demo;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.web.bind.annotation.GetMapping;
6 import org.springframework.web.bind.annotation.RestController;
7
8 /**
9  * Spring Boot Application - Equivalent to Python Flask App
10  *
11  * This Java Spring Boot application replicates the functionality of:
12  *
13  * Python Flask Code:
14  * =====
15  * from flask import Flask
16  *
17  * app = Flask(__name__)
18  *
19  * @app.route('/')
20  * def home():
21  *     return "Hello, World!"
22  *
23  * if __name__ == '__main__':
24  *     app.run(debug=True)
25  *
```

PYTHON TO JAVA FILE HANDLING CON...

Convert the following Python Flask API code into Java (Spring Boot).

Requirements:

- Maintain identical functionality
- Use appropriate Spring Boot annotations
- Ensure the code is executable

Also provide:

1. Explanation of the converted code
2. Differences in syntax and libraries
3. Performance comparison
4. Observations

Python Code:

```
from flask import Flask

app = Flask(name)

@app.route('/')
def home():
    return "Hello, World!"
```

OUTPUT:

Hello, World!

Task -5: (Basic Function Translation (Python to C++))

Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Instructions:

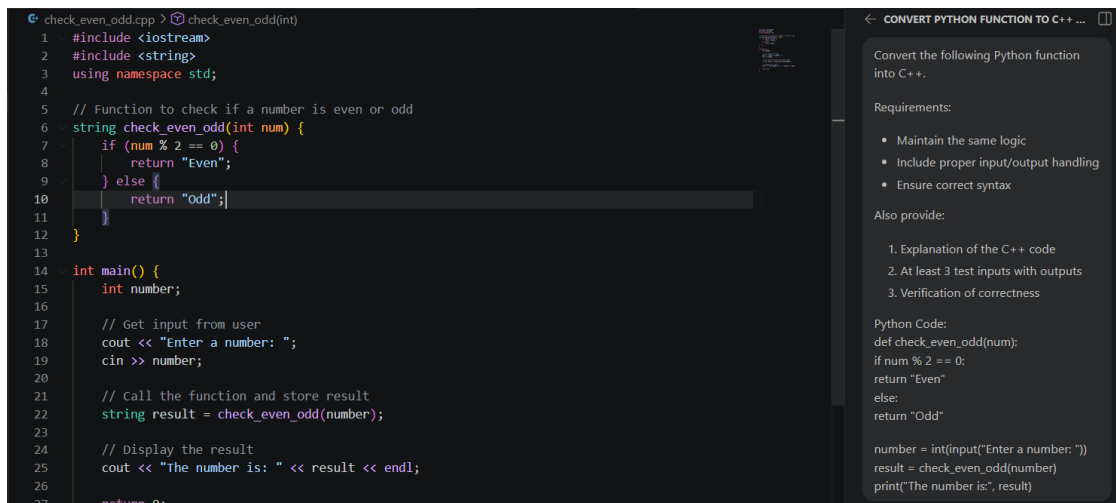
- Provide AI with a basic Python function that determines if a number is even or odd.
- Prompt AI to generate equivalent C++ code.
- Ensure correct syntax, input handling, and output formatting.
- Compile and run the C++ program to verify it produces the same result as the Python version.
- Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and

output and if required, screenshots.



```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Function to check if a number is even or odd
6 string check_even_odd(int num) {
7     if (num % 2 == 0) {
8         return "Even";
9     } else {
10        return "Odd";
11    }
12 }
13
14 int main() {
15     int number;
16
17     // Get input from user
18     cout << "Enter a number: ";
19     cin >> number;
20
21     // Call the function and store result
22     string result = check_even_odd(number);
23
24     // Display the result
25     cout << "The number is: " << result << endl;
26
27     return 0;
28 }
```

CONVERT PYTHON FUNCTION TO C++ ...

Convert the following Python function into C++.

Requirements:

- Maintain the same logic
- Include proper input/output handling
- Ensure correct syntax

Also provide:

1. Explanation of the C++ code
2. At least 3 test inputs with outputs
3. Verification of correctness

Python Code:

```
def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

number = int(input("Enter a number: "))
result = check_even_odd(number)
print("The number is:", result)
```

OUTPUT:

```
Enter a number: 4
The number is: Even
```